BRAINWARE UNIVERSITY

BRAINWARE UNIVERSITY

PYTHON PROJECT-BASED LEARNING REPORT (WEEK 3)

1. Basic Information

Title of the Project:	**Employee Management System & Library Management System**
Programs Covered:	**Program 1: Employee Management System** **Program 2: Library Management System**
Name(s) of Student(s):	**Soumyadwip Das (BWU/BTS/25/169)** **Ankita Paul (BWU/BTS/25/180)** **Sonia Naskar (BWU/BTS/25/195)** **Joyeta Mondal (BWU/BTS/25/214)** **Sayantan Bharati (BWU/BTS/25/503)**
Programme & Semester:	**B.Tech CSE & 3rd Semester**
Course Name & Code:	**Python Programming Lab (BES09013)**
Name of the Guide/Supervisor:	**MR. PRANAB GHARAI & MR. INDRANIL DAS**
Project Date	**14/08/2026**

2. Introduction and Background

Week 6 covers two console-based CRUD systems: an **Employee Management System** and a **Library Management System**. Both use dictionaries, validation, unique IDs, and menu-driven operations.

3. Objectives of the Project (Week 6)

Program 1 - Employee Management System

- Create a console-based employee management system.
- Generate unique employee IDs using random.
- Validate employee name, department, and salary.
- Implement add, update, delete, search, and list operations.
- Test valid and invalid inputs.

Program 2 - Library Management System

- Manage members, books, and issue/return records.
- Generate unique IDs for members, books, and issues.
- Validate member email and phone details.
- Implement member and book management.
- Prevent double-issuing books and deleting members with active loans.

4. Problem Statement / Driving Question

Program 1: How can employee records be managed with unique IDs and valid inputs?

Program 2: How can members, books, and loans be managed while preventing invalid operations?

5. Methodology (Week 6 Activities)

Program 1 - Employee Management System

- Reused box-drawing functions for the interface.
- Created a random, collision-free employee ID generator.
- Added validation for name, department, and salary.
- Implemented add, update, delete, search, and list functions.
- Tested invalid department and salary inputs.

Program 2 - Library Management System

- Reused the boxed interface and menu structure.
- Created a shared unique-ID generator for members, books, and issues.
- Added validation for member name, email, and phone.
- Implemented member and book management.
- Used a separate ISSUED dictionary for loan records.
- Prevented double-issuing books and deleting members with active loans.

6. Expected Outcomes / Deliverables (Week 6)

- Working Employee Management System with CRUD operations.
- Working Library Management System with member, book, and loan management.
- Validated input for required fields.
- Safeguards against invalid library operations.
- Consistent boxed console interfaces.
- Future improvements: file/database storage and due-date tracking.

7. Core Logic & Output (Program 1)

Program 1 - Unique ID Generation and Record Deletion

- generate_id() creates a unique random employee ID.
- delete_employee() checks if the employee exists before deleting.

```
def generate_id():
    """Generate a unique 4-digit employee ID, e.g. E1234."""
    while True:
        emp_id = "E" + str(random.randint(1000, 9999))
        if emp_id not in EMPLOYEES:
            return emp_id            # retry until a unique ID is found

def delete_employee(emp_id):
    if emp_id not in EMPLOYEES:
        print_boxed(["Employee not found."])
        return
    name = EMPLOYEES[emp_id]['name']
    del EMPLOYEES[emp_id]            # remove the record from memory
```

Program 1 - Employee Management System

8. Core Logic & Output (Program 2)

Program 2 - Shared ID Generation and Issue Records

- generate_id(prefix) creates unique IDs for members, books, and issues.
- issue_book() prevents duplicate issues and creates a separate loan record.

```
def generate_id(prefix):
    """Generate a unique ID like B1234, M5678, I9012."""
    while True:
        new_id = prefix + str(random.randint(1000, 9999))
        if new_id not in BOOKS and new_id not in MEMBERS and new_id not in ISSUED:
            return new_id            # unique across all three record types

def issue_book():
    book_id = input("> Enter book ID: ").strip().upper()
    if any(r["book_id"] == book_id for r in ISSUED.values()):
        print_boxed(["ERROR: Book is already issued."])
        return
    member_id = input("> Enter member ID: ").strip().upper()
    issue_id = generate_id("I")
    ISSUED[issue_id] = {"book_id": book_id, "member": member_id}   # new issue record
```

Program 2 - Library Management System

10. Tools & Technologies Used

Tool / Technology	Purpose
Python 3.x	Core programming language used to implement both console systems
Python Dictionaries	In-memory storage of employees, members, books, and issue records, each keyed by a unique ID
random module	Generating unique, prefix-based IDs for employees, members, books, and issue records
Input Validation Loops	Re-prompting until a valid department/salary, or a valid member email/phone, is provided
String Formatting (f-strings)	Rendering aligned tabular listings of employees, members, books, and issued loans
Functions & Control Flow	Modularising add/update/delete/search/issue/return operations across both systems

11. References

- Python Official Documentation: https://docs.python.org/3/
- Python random module Reference: https://docs.python.org/3/library/random.html
- Python Dictionaries - Data Structures Guide: https://docs.python.org/3/tutorial/datastructures.html
- Brainware University - Machine Learning Course Guidelines

12. Signatures

Signature of Student(s): **Date of Submission:**

Signature of Guide/Supervisor: